

ORIGIN: Google Street-view Image Location Discovery

Aadarsh Hegde
UIUC

Sairam Penumarthy
UIUC

Praneeth Rangamudri
UIUC

Abstract

Identifying the location at which different images have been taken from have long had applications in national defense and security, but in recent years, has also become increasingly popular in video games such as GeoGuessr. This project aims to translate the popular game to computers by using computer vision techniques to localize where different street-view images originate from instead of prompting a human user.

1 Introduction

GeoGuessr is a popular browser-based game developed by Swedish software engineer Anton Wallén. The game makes heavy use of the Google Maps API, using it to drop players in a random locations and asking them to guess precisely where they are. The game then assigns a score based on how close user's guesses are to the real location.

This project is an attempt at translating a simplified version of the game for computers. In other words, instead of having a human guess where an image originated from, we have the computer run algorithms and make a prediction as to where the image originated from. To simplify this process, we decided to have the computer provide a guess as to which country an image originated from rather than the precise latitude longitude coordinate that a human would normally guess with.

We make use of a Kaggle dataset containing GeoGuessr images and their corresponding country. We had initially planned for this project to use a custom-curated dataset filled with satellite-based imagery from EarthExplorer with up to 1000 images per country as seen in our progress report. This approach had been validated at the time by downloading test images from datasets available on the site. Unfortunately, we realized soon after that EarthExplorer does not have support for automated batch downloads (ie. we had to manually go to

every country, select a region, and download images one by one). This issue creates a hard cap on how many images we could realistically retrieve.

Due to this, we pivoted to using a pre-existing dataset containing street-view imagery that was grouped by the country they were taken in. Besides the change to our data, the structure of the rest of the project remained unchanged and is described in its entirety below.

2 Approach

At a high level, our approach remains the same as described in our progress report. The first step of this project involved data collection and pre-processing. As mentioned earlier, we were initially working towards a version of the project that would use satellite data gathered from EarthExplorer. While we were able to gather small subsets of data and construct small datasets, we found that the batch downloading that EarthExplorer provided only allowed batch downloading of the image metadata and not the images themselves. This meant that to construct our dataset, we had to manually download each image from a country one by one and this quickly became infeasible.

We chose to pivot to a pre-configured dataset consisting of 50k images from geoguessr grouped by their country of origin [1]. Unfortunately this meant that we had to abandon the satellite data we had collected and pivot to street-view data instead. However, this change allowed us to quickly get access to hundreds of images without having to construct a dataset ourselves. Below is an example of the change from the old data to the new data.



Figure 1: Old Satellite Imagery



Figure 2: Geoguessr Dataset Imagery Sample

Using the new Geoguessr data, we explore two complementary approaches for country-level image classification: classical computer vision-based feature extraction and convolutional neural network (CNN)-based feature extraction. Classical features, including edge, texture, and keypoint descriptors, provide an interpretable and computationally efficient baseline, while CNN-based representations may capture more nuanced information. We first evaluate the performance of classical features alone in order to establish a baseline and identify their strengths and limitations before introducing CNN-based features in subsequent experiments.

2.1 Classical Feature Extraction

For classical feature extraction, we implement four key functions: extract keypoints, extract edges, extract texture, and extract color histogram. Each function was chosen deliberately and the motivation for each is described in the following sections.

2.1.1 Keypoint Extraction

Often times, images in street view will capture similar landmarks at different scales/from different angles. For example, consider street view images in Paris. There are numerous streets that may capture views of the Eiffel Tower. We want to be able to identify when these landmarks are present so that when KNN runs, we have a good chance of the test image's feature vector being near feature vector's for France.

For implementing keypoint detection, we make use of SIFT keypoint extraction from the OpenCV library.

2.1.2 Edge Extraction

While keypoint extraction primarily targeted identifying common landmarks, the motivation behind edge extraction is to identify common architecture within countries. In many countries, the architectural design of buildings is drastically different. This is of course more apparent between developing and first world countries, but is still critical in distinguishing first world countries. This step is critical to gain knowledge on the geometry of the street we are viewing. Below is an example of the differences in edges between a training sample from Finland and a sample from Argentina.

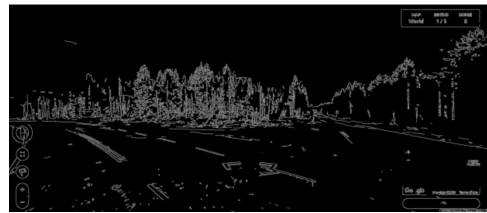


Figure 3: Finland Training Sample Edges



Figure 4: Argentina Training Sample Edges

We can see above that there is a clear geometrical difference in the trees that are in Finland and Argentina. These types of differences are what we hope the edge features capture.

2.1.3 Texture Extraction

Texture extraction is a critical step in distinguishing landscapes between countries. Some countries may be more trop-

ical and lush resulting in smoother textures, whereas more northern countries may have more mountains resulting in a more jagged texture [3]. This step ensures that these differences in images are captured and added to our feature vector.

2.1.4 Color Histogram

Finally, we also extract color information from each image. Often times, countries have drastically different color schemes. For example, countries like Costa Rica tend to have a more warm, vibrant color scheme. However, more northern countries like Finland and Denmark tend to have more grays and overall exhibit a colder color scheme.

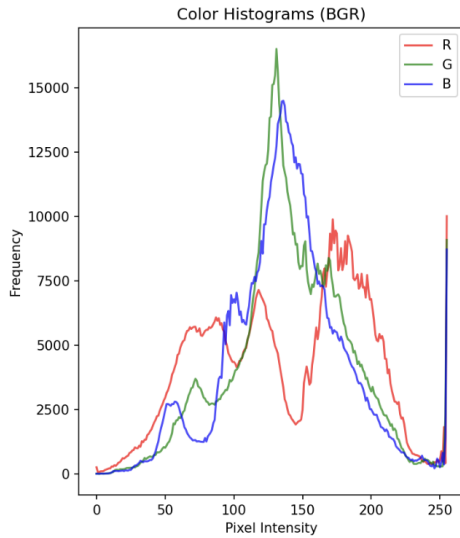


Figure 5: Finland Training Sample Color Histogram

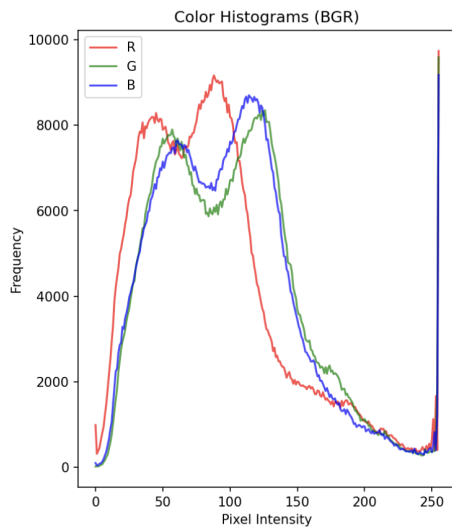


Figure 6: Costa Rica Training Sample Color Histogram

By extracting color features, we make sure to capture these differences in our overall feature vector, hopefully improving classification in the KNN phase.

2.1.5 Other Feature Extractions

A good question to ask is why not add more features to our feature vector (corner detection, etc.). The main tradeoff we consider is performance vs runtime. As we add more features to our feature vector, there are two primary costs.

One cost is simply the added computation to extract these new features. The other cost is caused by the curse of dimensionality which significantly slows down our KNN classification as we add more dimensions.

Due to this, we empirically evaluated when adding more features became meaningless. In other words, if you consider the tradeoff as a graph where the x-axis represents number of features and the y-axis represents accuracy, we found where performance stopped improving.

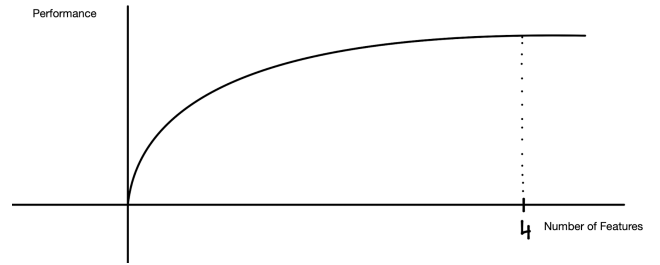


Figure 7: Tradeoff Graph Illustration

As seen in the graph, in our case we found the four feature extraction functions we chose captured the max performance we could get. Adding additional feature extractions like corner detection resulted in no meaningful performance gain and only increased runtime so these were omitted.

2.1.6 Combining Feature Extractions

Once all feature extraction algorithms have run, the results of all these functions are aggregated in a high dimensional feature vector. By design, our training algorithm creates one feature vector per training image. This creates a feature space which is then used during testing for the K-Nearest Neighbors algorithm which we describe in 2.3.

2.2 CNN Feature Extraction

CNN feature extraction follows a similar base outline as classical, except the features aren't extracted by manually

choosing the features and their algorithms, but instead using a trained neural net.

Instead of creating a custom NN topology, we decided to use a pretrained image classification model, since their second-to-last layer outputs its own feature vector we could use. However, due to this, we needed to find a model which was most compatible with our project goals, and so the first step is finding this.

Our second step involved optimizing this model on our data set, as they were pretrained on datasets which may have features vastly different from those one would normally see in a street-view image. Without this step, the model would put less weight on, say, a particular license plate on a car, and an unfairly large amount of weight on a cat on the street, which is not as useful when identifying location.

Finally, we can run our data through the newly trained feature extractor and obtain feature vectors from the second-to-last layer.

2.2.1 Choosing a Pre-trained Model

We needed to find a pre-trained model which, before any optimizations had occurred, had the highest accuracies during classification on our dataset. Two popular image classification models we tried were VGG16 and ResNet50. When testing, we found that ResNet50 had the best accuracies by default, and so chose this for our future training.

2.2.2 Custom Model Training

To train a custom model on top of ResNet50, we used a concept called "Transfer Learning" [2]. Effectively, we use the pretrained model's learnings as a foundation upon which we can build an image classifier to fit our needs, which are to identify countries from street-view images. To do this, we change the final, fully-connected layer to classify into the number of countries we are using. We also set the loss function to cross entropy and our optimizer to ADAM with a learning rate of 0.0001. Once all of this is done, we run a basic training loop for some number of epochs. We found that 5 epochs provided the most balanced results.

This training process takes a very long time. In fact, orders of magnitude longer than classical classification alone. However, once it's trained, we can save the model and use it whenever we please.

2.2.3 Running our Model and Extracting Feature Vector

Now that we have a CNN feature extractor model, we can use it to get our features. This involves, as mentioned before, taking the vector output just before the final layer is reached, which ends up being a 2048-dimensional output.

2.3 KNN Classification

To output our final classification, we utilized a K-Nearest Neighbors (KNN) classifier to perform country-level classification. We chose KNN because of its simplicity, interpretability, and suitability for similarity-based classification. KNN works by assigning a class label to a test sample based on the labels of its closest neighbors in feature space. Given a test image, its feature vector is compared against all training feature vectors using a distance metric, and the most frequent label among the k nearest samples is selected as the predicted class. In our implementation, we used Euclidean distance and distance-weighted voting, allowing closer neighbors to contribute more strongly to the final prediction. In order to ensure that all feature dimensions contribute comparably to the distance computation, we standardized feature vectors prior to classification.

2.4 Evaluation Methodology

For our classical feature extraction-based approach, we evaluated performance using three different subsets of countries in order to develop a minimal working product while assessing the robustness of the proposed pipeline. Each subset contains five distinct countries, allowing us to conduct controlled experiments with a manageable number of classes while varying geographic diversity and visual similarity. For each country, available images were divided into disjoint training and testing sets to enable fair evaluation of generalization performance.

Our first subset includes Cambodia, Colombia, Bolivia, Bulgaria, and Botswana. These countries were chosen deliberately with the goal of having all chosen countries distinct from each other. We wanted to have distinction between countries so that we could best evaluate how well our classifier was performing as we had the hunch that neighboring countries with similar geography could easily be mistaken for each other. Here is the breakdown of the total number of training images we used for each country in this set: Bolivia: 96, Botswana: 120, Bulgaria: 192, Cambodia: 96, Colombia: 228. Here is the breakdown of the total number of testing images we used for each country: Bolivia: 20,

Botswana: 24, Bulgaria: 25, Cambodia: 22, Colombia: 23

The second subset consists of Finland, Austria, Argentina, Bangladesh, and Costa Rica. This set introduces a broader mix of climates, terrains, and levels of urbanization, providing a more challenging and diverse evaluation scenario. Here is the breakdown of the total number of training images we used for each country: Argentina: 636, Bangladesh: 78, Costa Rica: 18, Finland: 1026, Austria: 258. Here is the breakdown of the total number of testing images we used for each country: Argentina: 53, Bangladesh: 28, Costa Rica: 9, Austria: 89, Finland: 23

The third subset includes Greece, Indonesia, Mexico, Luxembourg, and Poland. This final set combines countries with overlapping architectural styles, vegetation, and environmental features, allowing us to further assess how well our feature-based approach generalizes across visually similar regions. Here is the breakdown of the total number of training images we used for each country: Greece: 204, Indonesia: 252, Mexico: 864, Poland: 816, Netherlands: 504. Here is the breakdown of the total number of testing images we used for each country: Greece: 44, Indonesia: 36, Mexico: 37, Poland: 47, Netherlands: 75

We also evaluate on a fourth set which includes all countries from sets 1-3 in order to conduct a stress-test of sorts and make the classification problem more challenging.

3 Results

3.1 Classical Feature Extraction Results

Country Set1: Finland, Austria, Argentina, Bangladesh, Costa Rica

```

Accuracy: 0.4824561403508772
precision    recall  f1-score   support

  Bolivia      0.56      0.25      0.34        20
  Botswana     0.70      0.58      0.64        24
  Bulgaria     0.59      0.52      0.55        25
  Cambodia     0.46      0.27      0.34        22
  Colombia     0.34      0.74      0.47        23

 accuracy      0.53      0.47      0.48       114
 macro avg     0.53      0.47      0.47       114
weighted avg     0.53      0.48      0.48       114

Confusion matrix (rows=true, cols=pred):
[[ 5  2  2  1 10]
 [ 2 14  2  2  4]
 [ 0  0 13  4  8]
 [ 1  3  1  6 11]
 [ 1  1  4  0 17]]

```

Figure 8: Confusion matrix and classification metrics for Country Set 1

In the first country subset, the model achieved an overall accuracy of 48.2%. This is more than double the random baseline of 20% expected for a five-class classification problem. Botswana and Bulgaria had the highest F1-scores

of 0.64 and 0.55. This indicates that these countries contain relatively distinctive visual patterns that are well captured by the classical feature descriptors. On the other hand, Bolivia and Cambodia showed lower recall values of 0.25 and 0.27, which suggests that a large portion of their images were misclassified as other countries.

An interesting observation that we saw is that Columbia had recall of 0.74 but a low precision of 0.34. This indicates that while most Colombian images were correctly identified, many images from other countries were incorrectly predicted as Colombia. The confusion matrix supports this observation, showing many of the other countries (Bolivia, Cambodia, and Bulgaria) being classified into the Colombian class. This suggests that Colombia functions as a visually dominant category, likely due to the diversity of landscapes and visual features present in its imagery, which overlap with those of other countries in the subset.

Country Set 2: Finland, Austria, Argentina, Bangladesh, Costa Rica

```

Accuracy: 0.6584158415841584
precision    recall  f1-score   support

  Argentina    0.69      0.79      0.74        53
   Austria     0.97      0.70      0.81        89
  Bangladesh    0.62      0.29      0.39        28
  Costa Rica    0.00      0.00      0.00         9
   Finland     0.33      0.91      0.48        23

 accuracy      0.66      0.66      0.66       202
 macro avg     0.52      0.54      0.48       202
weighted avg     0.73      0.66      0.66       202

Confusion matrix (rows=true, cols=pred):
[[42  1  3  0  7]
 [10 62  1  0 16]
 [ 5  1  8  0 14]
 [ 2  0  1  0  6]
 [ 2  0  0  0 21]]

```

Figure 9: Confusion matrix and classification metrics for Country Set 2

The second country subset achieved the highest overall accuracy at 65.8%, representing the strongest performance across all experiments. Austria and Argentina had the highest F1-scores of 0.81 and 0.74. Austria also had really high precision (0.97) along with high recall (0.70), while Argentina showed a balanced precision–recall profile. This indicates that classical features are effective at capturing distinctive visual cues associated with these regions, such as architectural patterns and environmental textures.

However, performance across classes remained uneven. Finland achieved very high recall (0.91) but relatively low precision (0.33). This indicates that while most Finnish images were correctly identified, many images from other countries were incorrectly classified as Finland. We also noticed that Costa Rica achieved zero precision and recall, reflecting a complete failure to correctly classify its limited

number of test samples. This highlights the sensitivity of similarity-based classifiers to small sample sizes and class imbalance. Overall, the confusion matrix reveals notable cross-class confusion, particularly among Argentina, Austria, and Finland, despite the higher aggregate accuracy.

Country Set 3: Greece, Indonesia, Mexico, Netherlands, Poland

Accuracy: 0.3472803347280335				
	precision	recall	f1-score	support
Greece	0.27	0.09	0.14	44
Indonesia	0.47	0.25	0.33	36
Mexico	0.26	0.46	0.33	37
Netherlands	0.49	0.29	0.37	75
Poland	0.33	0.66	0.44	47
accuracy			0.35	239
macro avg	0.36	0.35	0.32	239
weighted avg	0.38	0.35	0.33	239
Confusion matrix (rows=true, cols=pred):				
[[4 4 16 8 12]				
[1 9 10 6 10]				
[3 1 17 4 12]				
[5 5 13 22 30]				
[2 0 9 5 31]]				

Figure 10: Confusion matrix and classification metrics for Country Set 3

The third country subset achieved the lowest accuracy of 34.7%, which, while still above the random baseline of 20%. This represents a noticeable decline in performance compared to the first two subsets. Poland achieved the highest recall (0.66) and F1-score (0.44), indicating comparatively stronger separability for this class. In contrast, Greece and Indonesia performed poorly, with Greece exhibiting particularly low recall (0.09) and Indonesia achieving a recall of 0.25, suggesting that images from these countries were frequently misclassified.

The confusion matrix for this subset reveals substantial misclassification across all classes, particularly among Greece, Indonesia, and Mexico, which are often confused with one another. The Netherlands achieved relatively high precision (0.49) but low recall (0.29), indicating that the model was conservative when predicting this class and frequently failed to identify Dutch images correctly. Overall, these results point to significant visual overlap among countries in this subset, highlighting the limitations of classical edge-, texture-, and keypoint-based descriptors when applied to visually heterogeneous and semantically complex regions.

Comparing results across the three country subsets reveals clear trends. Country Set 2 yielded the highest accuracy, likely due to the presence of visually distinctive countries

with stronger architectural or environmental cues. Country Set 1 demonstrated decent performance, with specific classes dominating predictions due to visual overlap. Country Set 3 proved the most difficult, suggesting that classical features alone are insufficient for distinguishing countries with similar visual characteristics or high intra-class variability. Overall, these experiments demonstrate that classical computer vision features can serve as a meaningful baseline for country-level image classification, consistently outperforming random chance across all subsets. However, the variability in performance and frequent confusion between visually similar regions underscore the limitations of low-level feature representations. These findings motivated the exploration of more expressive, learned feature representations to capture nuances that classical feature extraction missed.

Country Set 4: Combination of Countries from Sets (1-3)

Accuracy: 0.2828828828828829				
	precision	recall	f1-score	support
Argentina	0.38	0.43	0.40	53
Austria	0.40	0.46	0.43	89
Bangladesh	0.40	0.14	0.21	28
Bolivia	0.40	0.10	0.16	20
Botswana	0.20	0.20	0.20	24
Bulgaria	0.31	0.20	0.24	25
Cambodia	0.30	0.14	0.19	22
Colombia	0.21	0.22	0.21	23
Costa Rica	0.00	0.00	0.00	9
Finland	0.16	0.61	0.25	23
Greece	0.30	0.07	0.11	44
Indonesia	0.19	0.14	0.16	36
Mexico	0.18	0.38	0.24	37
Netherlands	0.25	0.17	0.20	75
Poland	0.16	0.38	0.22	47
accuracy			0.28	555
macro avg	0.32	0.25	0.24	555
weighted avg	0.37	0.28	0.29	555
Confusion matrix (rows=true, cols=pred):				
[[23 1 1 0 3 1 2 2 0 4 1 3 5 3 4]				
[2 41 0 0 2 0 0 0 5 3 3 14 9 9]				
[2 0 4 0 0 0 0 3 0 3 0 3 0 13]				
[3 0 0 2 0 0 0 0 3 0 3 0 1 4 3 11]				
[3 0 0 0 7 0 0 0 0 2 0 0 0 1 6]				
[4 0 0 0 0 5 1 0 0 3 0 0 3 2 7]				
[3 0 1 0 0 0 0 2 0 0 1 2 3 0 6]				
[2 0 0 0 0 0 0 0 2 0 2 0 2 2 4]				
[0 0 0 0 0 0 0 0 0 1 0 2 0 1 5]				
[0 0 0 0 0 0 1 0 14 0 0 2 2 4]				
[2 3 1 1 0 2 3 0 7 3 0 9 5 6]				
[4 0 0 1 0 0 1 0 0 6 0 5 8 4 7]				
[2 0 2 0 1 0 1 0 4 1 14 3 7]				
[5 0 1 0 0 3 0 1 0 18 1 4 5 13 18]				
[1 0 0 0 0 2 1 3 0 13 0 0 5 4 18]]				

Figure 11: Confusion matrix and classification metrics for Country Set 4

Finally, we conducted an additional stress-test by combining images from all three subsets into a single 15-class classification task to further evaluate scalability. We achieved an overall accuracy of 28.3%, which, while substantially above the random baseline of approximately 6.7%, represents a notable degradation in performance compared to the smaller, controlled subsets. Classes such as Austria and Argentina maintained relatively strong performance, whereas many countries exhibited low recall or collapsed entirely, with Costa Rica achieving zero precision and recall.

Overall, these results demonstrate that classical computer vision features provide a meaningful baseline for country-level image classification, consistently outperforming random chance. However, the sharp decline in performance as class diversity increases highlights fundamental limitations in edge, texture, and keypoint-based representations, and represents a need for a more nuanced approach.

3.2 CNN Feature Extraction Results

Country Set1: Finland, Austria, Argentina, Bangladesh, Costa Rica

```
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -train 1
--- Starting CNN Feature Extraction ---
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -test 1
--- Starting CNN Feature Extraction ---
Accuracy: 0.6842185263157895
precision recall f1-score support
Bolivia      0.56  0.45  0.50    20
Botswana     1.00  0.21  0.83    24
Bulgaria     0.70  0.76  0.73    25
Cambodia     0.68  0.68  0.68    22
Colombia     0.56  0.78  0.65    23
accuracy     0.70  0.68  0.68   114
macro avg    0.70  0.68  0.68   114
weighted avg 0.71  0.68  0.69   114
Confusion matrix (rows=true, cols=pred):
[[ 9  0  3  2  6]
 [ 0 17  2  4  1]
 [ 3  0 19  0  3]
 [ 2  0  1 15  4]
 [ 2  0  2  1 18]]
```

Figure 12: Confusion matrix and classification metrics for Country Set 1

Country set 1 shows an overall accuracy of 0.68, which is quite good considering the baseline of 0.2 when choosing randomly. Botswana and Bulgaria lead in F1 scores over the other countries, but the rest of the countries are not far behind, indicating feature extraction is balanced among countries, but still fairly accurate in recall and precision.

Where generally, countries had higher precision than recall, Colombia and Bulgaria had the opposite, with Bulgaria having a massive difference in the opposite direction.

All of this follows the overall distribution and proportions of classical feature extraction's confusion matrix and results, but with higher scores across the board. This is a trend which continues in the following sets, and many of the explanations for overall relationships between scores of countries in classical extraction applies to the CNN as well.

Country Set 2: Finland, Austria, Argentina, Bangladesh, Costa Rica

```
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -train 2
--- Starting CNN Feature Extraction ---
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -test 2
--- Starting CNN Feature Extraction ---
Accuracy: 0.8514851485148515
precision recall f1-score support
Argentina    0.83  0.94  0.88    53
Austria      0.97  0.85  0.91    89
Bangladesh   1.00  0.54  0.70    28
Costa Rica   0.44  0.89  0.59    9
Finland      0.74  1.00  0.85    23
accuracy     0.80  0.84  0.85   282
macro avg    0.80  0.84  0.85   282
weighted avg 0.89  0.85  0.85   282
Confusion matrix (rows=true, cols=pred):
[[50  1  0  1  1]
 [ 3 76  0  3  7]
 [ 0 11  0  0  0]
 [ 1  0  0  8  0]
 [ 0  0  0  2 23]]
```

Figure 13: Confusion matrix and classification metrics for Country Set 2

Country set 2 performed the best out of all data sets at an accuracy of 0.85. High precision and recall is present among all countries except for Bangladesh and Costa Rica. Bangladesh has perfect precision, but 0.54 recall, indicating CNN extraction never assigns Bangladesh to a non-Bangladesh feature vector, but fails to identify all true Bangladesh countries. Meanwhile, Costa Rica has a competitive recall of 0.89, but low precision at 0.44.

Positively, Bangladesh and Costa Rica, which had very low precision and recall (with Costa Rica even having 0 F1 score), achieved much higher F1 scores with a CNN.

Country Set 3: Greece, Indonesia, Mexico, Netherlands, Poland

```
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -train 3
--- Starting CNN Feature Extraction ---
(cs543) psairam@sairams-MacBook-Pro cv-geoguessr % python3 geoguessr.py -cnn -test 3
--- Starting CNN Feature Extraction ---
Accuracy: 0.6903765690376569
precision recall f1-score support
Greece       0.91  0.48  0.63    44
Indonesia    0.91  0.26  0.69    36
Mexico       0.50  0.89  0.64    37
Netherlands  0.88  0.68  0.77    75
Poland       0.57  0.85  0.68    47
accuracy     0.75  0.69  0.69   239
macro avg    0.75  0.69  0.69   239
weighted avg 0.77  0.69  0.69   239
Confusion matrix (rows=true, cols=pred):
[[21 17  2  3]
 [ 0 20 11  3  2]
 [ 0  0 33  0  4]
 [ 0  1  2 51 21]
 [ 2  0  3  2 40]]
```

Figure 14: Confusion matrix and classification metrics for Country Set 3

With an accuracy of 0.69, country set 3 performed admirably. F1 scores were very balanced here with a total range of 14, even moreso than set 1, and were on par with the mean of it as well. Clearly these results were more consistent if still even on average.

However, there are still many inconsistencies between precision and recall among most countries. Only the Netherlands maintained good values for both, with 0.88 precision and 0.68 recall. Interestingly, this is a stark improvement compared to Netherlands in the classical extraction. Greece and Indonesia had precision values both at 0.91, but recalls of 0.48 and 0.56 respectively. Conversely, Mexico and Poland had precision values of 0.50 and 0.57, but recalls of 0.89 and 0.85 respectively.

Country Set 4: Combination of Countries from Sets (1-3)

```

(cs543) psairam@psairam-MacBook-Pro: cv-geoguessr % python3 geoguessr.py -cnn -test 4
--- Starting CNN Feature Extraction ---
Accuracy: 0.37117117117117115
precision  recall  f1-score  support
Argentina  0.39    0.49    0.44    53
Austria    0.65    0.83    0.73    89
Bangladesh 0.47    0.11    0.09    28
Bolivia    1.00    0.05    0.10    20
Botswana   0.40    0.58    0.47    24
Bulgaria   0.50    0.08    0.14    25
Cambodia   0.44    0.18    0.26    22
Colombia    0.41    0.30    0.35    23
Costa Rica  0.00    0.00    0.00     9
Finland    0.56    0.43    0.49    23
Greece     0.56    0.32    0.41    44
Indonesia  0.21    0.25    0.23    36
Mexico     0.12    0.05    0.08    37
Netherlands 0.73    0.15    0.24    75
Poland     0.19    0.62    0.29    47

accuracy   0.37    0.37    0.37    555
macro avg  0.42    0.30    0.29    555
weighted avg 0.46    0.37    0.35    555

Confusion matrix (rows=true, cols=pred):
[[26 20  1  0  0  1  0  1  0  1  0  1  0  0  2]
 [ 7 74  0  0  0  0  1  0  3  0  0  0  0  4]
 [12  4  3  0  0  0  0  3  0  2  0  3  0  1]
 [ 4  0  7  1  6  0  0  1  0  0  0  0  0  1]
 [ 3  1  3  0 14  0  2  0  0  1  0  0  0  0]
 [ 3  1  8  0  7  2  1  2  0  1  0  0  0  0]
 [ 3  0 12  0  3  0  4  0  0  0  0  0  0  0]
 [ 3  0  5  0  5  1  2  7  0  0  0  0  0  0]
 [ 0  3  2  0  0  0  0  0  0  0  3  0  0  1]
 [ 1  7  0  0  0  0  0  1  0 10  0  1  0  3]
 [ 1  1  0  0  0  0  0  0  0 14  7  5  0 16]
 [ 2  1  0  0  0  0  0  0  0  0  9  0  0 24]
 [ 1  0  0  0  0  0  0  1  0  0  6  7  2 19]
 [ 0  1  0  0  0  0  0  0  0  0  3  5  4 11]
 [ 0  1  0  0  0  0  0  0  0  2  7  5  3 29]]

```

Figure 15: Confusion matrix and classification metrics for Country Set 4

Just as we did for classical, an aggregate of all images was used for a fourth country set. Since the CNN workflow functions differently than classical, as the feature extractor itself needs to be trained on data, this extractor was trained on all pieces of data to correctly capture features across all classes.

You will first see that the overall accuracy is much lower than individual country sets, sitting at 0.37. While still good, and even better than classical, it pales in comparison to individual datasets. It also follows the same trends as the individual datasets: countries which performed well equally performed in this larger set; those that suffered, however, suffered worse here. For example, Austria and Argentina, with Austria especially, continued to lead in F1 score. Costa Rica, which exhibited poor precision and recall before, now sits at 0 in total. Clearly, a larger dataset substantially hurts efficacy of the extractor.

4 Discussion and Conclusion

As we can see from the results, CNN-based feature extraction outperforms classical feature extraction across the board. Interestingly, countries generally performed equally well relative to other countries across both methods, indicating the features extracted were similar, but likely more detailed in the CNN.

4.1 Failure Point Analysis

One of the main reasons the accuracy for classical-based feature extraction may be lower than the CNN-based

extraction is due to the nature of the training data.

Unlike raw google maps street-view images, geoguessr street-view images contain some UI overlay to facilitate the game. Notice in figure 2 the arrows on the ground, the compass and navigational tools on the left side, and other UI specifics on the top right. The main thing to note is that these features are all consistent regardless of the country an image is from, which is a problem because it may affect KNN classification with artificially nearby neighbors.

Another issue arising from the training data is data sparsity. The dataset we make use of is not consistent with how many images each country has (ie. Brazil has over 2k images and Costa Rica only has 30). This sparseness seems to be affecting results for classical-based feature extraction. For example, Costa Rica from Country Set 2 had 0% accuracy and this is likely due to the fact that this country only had about 30 images in the dataset. Even in the CNN-based feature extraction, Costa Rica exhibited the worst precision due to the lack of training data available. Adding more training data would likely improve performance across the board.

4.2 Runtime

Runtime is very important when considering image processing, considering how fast it can scale with larger and more complex image datasets.

We found that, when training the KNN classifier on datasets 1-3 with classical features and without cuda, it took 624 seconds to process everything. When testing, it took 61 seconds, for a total of 685 seconds. On the other hand, doing the same but using CNN features took 369 seconds, and testing took 47 seconds for a total of 416 seconds. This is a 39% improvement over classical. It's worth noting that training the feature extractor takes much, much longer, especially without a GPU. With a GPU, it takes approximately 20 minutes. However, this is negligible for the CNN, since the model only needs to be generated once and then reused. Clearly, the CNN extractor exhibited much better runtimes than classical.

4.3 Conclusion

It is clear based on the CNN extractor's superior accuracy and runtime that it outperforms the classical extractor effectively in all regards. However, our process can be improved by future work. Namely, obtaining a more robust dataset with many countries could improve our issues of data sparsity. Also, our CNN extractor can be drastically

improved by identifying things like license plates, the side the camera source is driving, and the sun's position. Details like this directly correlate to a country and can narrow down possibilities substantially instead of only doing normal feature extraction. The use cases for an efficient geolocator like this are numerous, and knowing a neural net can beat standard algorithms opens the door to further study.

5 Individual Contributions

Aadarsh: Responsible for all classical feature extraction work in the code including creating feature vectors, and saving the feature space for input into KNN.

Primary writer of the abstract, introduction, and approach (up to section 2.1.6). Contributor of discussion section.

Sairam: Built and trained our custom ResNet50 CNN feature extractor. Optimized it to work best on the type of images our project was handling.

Wrote out the CNN Feature Extraction approach and results. Contributed to the discussion section.

Praneeth: Worked on KNN classifier implementation for classical feature extraction and wrote the results for the classical feature extraction. Also organized training and test data sets.

References

- [1] GeoLocation - Geoguessr Images (50K) — kaggle.com. <https://www.kaggle.com/datasets/ubitquitin/geolocation-geoguessr-images-50k/code>. [Accessed 18-12-2025].
- [2] Sasank Chilamkurthy. Transfer Learning for Computer Vision Tutorial. https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html, 2017. [Accessed 18-12-2025].
- [3] Trapti Kalra. Texture Analysis with Deep Learning for improved Computer Vision — trapti.kalra. <https://medium.com/@trapti.kalra/texture-analysis-with-deep-learning-for-improved-computer-vision-aa627c8bb133#b9ea>. [Accessed 18-12-2025].